

Автоматический мониторинг и классификация инструментов пентеста по MITRE ATT&CK

Этап 4: Демонстрация проекта

Новосельский Д • Коломийцев Н • Толмачёв Н • Ширинов А • Игнатов Д

2026

Вступление

Команда завершает **четвёртый, финальный этап** проекта **OffensiveToolMapper** — системы автоматического мониторинга и классификации инструментов пентеста по MITRE ATT&CK.

В мире кибербезопасности специалисты — *Red Team*, *аналитики SOC*, *исследователи угроз* — работают с постоянно меняющимся набором атакующих инструментов:

- На **GitHub** ежедневно появляются десятки новых репозиториев
- На **Exploit-DB** и **PacketStorm** выходят свежие эксплойты
- Понять, какую **технику MITRE ATT&CK** реализует инструмент — необходимо для приоритета в обороне

Решение: OffensiveToolMapper

Система решает задачу **автоматически**:

1. **Собирает** инструменты из GitHub, RSS-лент и PacketStorm
2. **Оценивает** нейросетью через вызов инструментов по команде LLM с JSON-схемой
3. **Строит** структурированную карту покрытия MITRE ATT&CK
4. **Предоставляет** результаты через веб-интерфейс и MCP-сервер для AI-агентов

Проблема и цель проекта

Проблема

- Ручной маппинг: **10–15 мин** на инструмент
- **50+ новых** инструментов в неделю
- Несколько рабочих дней только на классификацию
- Нет стандартизации: «lateral movement», «LM», «перемещение по сети»
- Данные несопоставимы между командами

Цель

Автоматизировать процесс от сбора до классификации:

1. Автоматический сбор из нескольких источников
2. Умная предфильтрация кандидатов
3. Структурированная LLM-классификация
4. Живая база данных с MITRE-маппингами

Решение в три шага

Автоматический сбор

Система находит новые инструменты из GitHub по ключевым словам, читает RSS-ленты и PacketStorm

Умная предфильтрация

Эвристическая оценка `pre_llm_score` отсеивает учебные проекты и форки без звёзд

Структурированная классификация

LLM через вызов инструментов по команде возвращает JSON: категория, тактики, T-коды, уверенность

Архитектура системы

СЛОЙ СБОРА ДАННЫХ (ETL)

etl_github.R | etl_rss.R | etl_packetstorm.R — унифицированный тиббл: name, url, description, source, stars, collected_at

СЛОЙ ОБРАБОТКИ (Pipeline)

pipeline.R — оркестрация: collect → normalize → assessment → refine_mitre → visualize, атомарные этапы с run_id

СЛОЙ ХРАНЕНИЯ (DuckDB)

inst/extdata/tools.duckdb — колоночная аналитическая БД, встраивается в R-процесс, полноценный SQL

СЛОЙ ВИЗУАЛИЗАЦИИ (Shiny + RDS)

RDS-снапшоты + reactivePoll — исключение конфликтов блокировок между пайплайном и UI

СЛОЙ ИНТЕГРАЦИИ (MCP-сервер)

mcp_server.R — 5 инструментов для AI-агентов: search_tools, get_tool_ttps, get_technique_tools, get_statistics, classify_new_tool

Источники данных



GitHub

Поиск по 18 ключевым словам:

exploit, payload, shellcode, c2, command-and-control, post-exploitation, lateral-movement, privilege-escalation, evasion, implant, RAT, backdoor, keylogger, rootkit, dropper, stager, beaconing, exfiltration

Фильтры:

• Языки: Python, Go, C, C++, PowerShell, Assembly, Rust

5000 запросов/час через GITHUB_PAT



RSS-ленты

3 ленты по умолчанию:

- **Exploit-DB** — эксплойты с CVE
- **The Hacker News** — обзоры инструментов
- **BleepingComputer** — уязвимости

Настройка:

- OTM_RSS_FEEDS — свои URL через «;»
- disabled — отключить RSS

Каждый item: заголовок, описание, ссылка → текст для LLM



PacketStorm

Архив эксплойтов с метаданными:

- CVE-идентификатор
- Платформа
- Тип атаки
- Автор и дата

2 режима доступа:

- API (PACKETSTORM_API_SECRET)
- Ручные URL (PACKETSTORM_URLS)

Парсинг через rvest по CSS-селекторам

Пайплайн: пять этапов обработки

01

collect

Параллельный вызов трёх ETL-модулей

Объединение через `bind_rows()`

Выход: таблица кандидатов с полями `name`, `url`, `description`, `source`, `stars`

02

normalize

Дедупликация по URL

Очистка текста: `str_squish`, `str_trim`

Подсчёт `pre_llm_score`:
ключевые слова, язык, тех. сигналы

03

assessment

LLM-классификация через

вызов инструментов по команде LLM

JSON-схема: `category`, `mitre_tactics`, `mitre_techniques`, `confidence_score`

Результаты → таблица `assessments`

04

refine_mitre

Второй LLM-проход для записей с высоким `confidence` ≥ 0.7

Добавление субтехник: T1059.001 вместо T1059

Флаг `is_refined` = TRUE

05

visualize

Построение 3 RDS-файлов из DuckDB

`reactivePoll` следит за `mtime`

Нет конфликтов блокировок с Shiny

LLM-интеграция: 7 провайдеров

Вызов инструментов по команде LLM вместо промптинга

Нейросеть **обязана** вызвать функцию с аргументами строго по JSON-схеме. Ошибки формата снижены с **-15%** до **<1%**.

JSON-СХЕМА КЛАССИФИКАЦИИ

- `category` — 8 значений (exploit, recon, c2...)
- `mitre_tactics` — массив названий тактик
- `mitre_techniques` — T-коды ["T1059.001"]
- `confidence_score` — уверенность (0–1)
- `detail_score` — полнота покрытия (0–1)
- `justification` — обоснование решения

7 провайдеров — единый интерфейс

Провайдер	Модель	Особенности
OpenAI	gpt-4.1-mini	Высокая точность, хороший вызов инструментов по команде LLM
Anthropic Claude	claude-sonnet-4-6	Лучший в MITRE-маппингах, субтехники
DeepSeek	deepseek-chat	Дешёвый, стабилен в JSON-выводе
Groq	llama-3.3-70b	Бесплатный тариф, минимальная латентность
Ollama	qwen3:8b	Полностью локальный, данные не уходят в сеть

Система скоринга и фильтрации

Pre-LLM скоринг (эвристика)

- **Ключевые слова:** 5 баллов каждое, max 30
- **Язык:** C/C++ 20, Asm 25, Go/Rust 18, Py 10
- **Тех. сигналы:** Makefile +5, .asm +10, бинарники +8

Порог: 25/100 — ниже не отправляются в LLM

OTM_PRE_LLM_THRESHOLD

Post-LLM скоринг (качество)

- **confidence_score** — уверенность (0–1)
- **detail_score** — полнота покрытия
- **Уникальные тактики:** +0.05 каждая, max 0.3

Итоговый рейтинг:

$\text{composite} = 0.4 \times \text{pre_llm} + 0.6 \times (\text{conf} \times 0.5 + \text{detail} \times 0.5)$

По нему сортируются карточки в браузере

Эффективность двухступенчатой системы

2–3×

экономия API-бюджета за счёт pre-LLM
фильтрации

60%

GitHub-репозитории — учебные
проекты (отсеяны)

<1%

ошибок формата с function calling (vs
–15%)

7

LLM-провайдеров через единый
интерфейс

Хранилище DuckDB и интерфейс Shiny

DuckDB — встраиваемая колоночная БД

Таблица	Назначение
candidates	Сырые и нормализованные записи (UUID, name, url, source, stars, pre_llm_score)
assessments	Результаты LLM-классификации (category, confidence, detail, justification)
mitre_mappings	Нормализованные TTP (tactic, technique_id, technique_name, is_refined)
pipeline_runs	История запусков (run_id, started_at, n_collected, n_assessed)
batch_summary	Агрегированная статистика по пакетным запускам

Shiny — 5 вкладок

Обзор KPI + тепловая карта	Инструменты Фильтры + карточки
Аналитика MITRE-матрица + слайдер	Уточнение Кандидаты на refine_mitre
Пайплайн — история запусков, кнопка запуска из UI, лог в реальном времени	

АРХИТЕКТУРНОЕ РЕШЕНИЕ: DUCKDB + RDS-СНАПШОТЫ

- **Пайплайн пишет** в DuckDB — полноценный SQL, оконные функции, в 3–4× быстрее SQLite
- **Visualize читает** DuckDB и строит RDS-снапшоты — заранее агрегированные данные
- **Shiny читает только RDS** — нет конфликтов блокировок с работающим пайплайном
- **reactivePoll** следит за mtime RDS-файлов — UI обновляется только при реальных изменениях

Что появилось или было доделано на этапе 4

7 LLM-провайдеров

Единый интерфейс `llm_provider.R` — смена провайдера без изменения кода, 3 строки в `.env`

RSS + PacketStorm

Два новых ETL-модуля. Гибкая настройка через `.env`: `OTM_RSS_FEEDS` и `PACKETSTORM_URLS`

Запуск из UI

Кнопка запуска пайплайна из Shiny, фоновое выполнение через `system2()`, лог в реальном времени

Мастер настройки

7 интерактивных шагов: провайдер, модель, API-ключ, GitHub PAT, лимиты, RSS, PacketStorm

Docker Compose

Два сервиса: `shiny-app` (порт 8788) и `mcp-server` (порт 3000). `docker compose up --build` — одной командой

reactivePoll

Обновление UI только при реальных изменениях RDS (проверка `mtime`), вместо `invalidateLater` каждые 10 сек

ДОПОЛНИТЕЛЬНО

- `run_collect_only.R` — скрипт без LLM для проверки источников (бесплатно)
- **CSS-правки** — белый фон в модальном окне, уменьшен `hero`-заголовок, читаемые цвета вкладок
- **MCP-сервер** — 5 инструментов для AI-агентов: поиск, TTP, статистика, классификация

Итоги и дальнейшие планы

РЕЗУЛЬТАТЫ

- Пайплайн из **5 этапов**: collect → normalize → assessment → refine_mitre → visualize
- **7 LLM-провайдеров** через единый интерфейс
- **3 источника** данных: GitHub, RSS, PacketStorm
- **Shiny-приложение** с 5 вкладками и кнопкой запуска
- **MCP-сервер** с 5 инструментами для AI-агентов
- **Docker Compose** для воспроизводимого развёртывания
- Двухступенчатая система **скоринга**

ПЛАНЫ

- **Human-in-the-loop** — ручное подтверждение маппингов, статус reviewed
- **LLM-обзор** — метрики: задержка, retry, стоимость по провайдерам
- **Экспорт в STIX/TAXII** — интеграция с SIEM и TI-платформами

КЛЮЧЕВЫЕ ВЫВОДЫ

1. Pre-LLM фильтрация **критически важна** — сокращает расходы на API в 2–3 раза
2. Вызов инструментов по команде LLM **надёжнее** промптинга: -15% → <1% ошибок
3. DuckDB + RDS-снапшоты — **архитектура** без блокировок
4. MCP **открывает новые сценарии** интеграции с AI-агентами

Спасибо за внимание!